Software Engineering

Lab 6

Vedant Sharma

PES2UG24CS577

Section 5-J

02-11-26

Code Coverage

```
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab6> coverage run -m pytest CodeCoverageTest_PES2UG24CS577_electricity_bill.py
============================= test session starts ==============================
platform win32 -- Python 3.12.6, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Vedant\PESU\Semester5\SE\Lab\Lab6
collected 6 items

CodeCoverageTest_PES2UG24CS577_electricity_bill.py ......

============================== 6 passed in 0.10s ==============================
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab6> coverage report -m
Name                                                Stmts   Miss  Cover   Missing
---------------------------------------------------------------------------------
CodeCoverageTest_PES2UG24CS577_electricity_bill.py     20      0   100%
electricity_bill.py                                    32      0   100%
---------------------------------------------------------------------------------
TOTAL                                                  52      0   100%
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab6>
```

Branch Coverage

```
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab6> coverage run --branch -m pytest BranchCoverageTest_PES2UG24CS577_electricity_bill.py
============================= test session starts ==============================
platform win32 -- Python 3.12.6, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Vedant\PESU\Semester5\SE\Lab\Lab6
collected 15 items

BranchCoverageTest_PES2UG24CS577_electricity_bill.py ...............

============================== 15 passed in 0.08s ==============================
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab6> coverage report -m
Name                                                Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------------------------
BranchCoverageTest_PES2UG24CS577_electricity_bill.py     35      0      0      0   100%
electricity_bill.py                                      32      0     22      0   100%
-----------------------------------------------------------------------------------------------
TOTAL                                                    67      0     22      0   100%
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab6>
```

Commits

```
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab6> git log
commit cfe9530b6e06fc0afb363e6f10f1628b49afa444 (HEAD -> main, origin/main, origin/HEAD)
Author: Vedant Sharma <vedantsharma069@gmail.com>
Date:   Tue Nov 3 00:09:04 2026 +0530

    Branch coverage test case update

commit 869d36ec00b60f2cd556df97fd3aec252e3df2a6
Author: Vedant Sharma <vedantsharma069@gmail.com>
Date:   Tue Nov 3 00:05:17 2026 +0530

    Code Coverage Test Case update

commit cfefc4cbcb8cdb844833aa901726b13b8dd08139
Author: Vedant Sharma <vedantsharma069@gmail.com>
Date:   Mon Nov 2 23:52:22 2026 +0530

    Initial Commit
```

Analysis

Phase 1: Code (Line) Coverage Achieving 100% line coverage confirms that the test suite executes every single line of the application code at least once. While it ensures no code is entirely unreached, this metric can create a false sense of security by ignoring alternate logical pathways and hidden edge cases.

Phase 2: Branch Coverage Branch coverage requires evaluating every control structure to both its True and False outcomes. Achieving 100% branch coverage demonstrates rigorous testing of all possible logical pathways and boundary conditions, forcing the inclusion of scenarios that line coverage misses and ensuring comprehensive test completeness.

Code

CodeCoverageTest_PES2UG24CS577_electricity_bill.py

```
import pytest

from electricity_bill import energy_charge, late_fee, subsidy



# Example test cases:

def test_low_usage_domestic():

    assert energy_charge(50, "domestic") == 200



def test_paid_on_time():

    assert late_fee(500, 0) == 0



# Minimum tests to hit every line of code (Line Coverage)

def test_energy_charge_errors():

    with pytest.raises(ValueError, match="unknown connection"):

        energy_charge(50, "industrial")

    with pytest.raises(ValueError, match="invalid units"):

        energy_charge(-10, "domestic")



def test_energy_charge_slabs():

    assert energy_charge(200, "domestic") == 850

    assert energy_charge(400, "commercial") == 4350



def test_late_fee_penalties():

    assert late_fee(500, 10) == 50

    assert late_fee(2500, 20) == 225



def test_subsidy_benefits():

    assert subsidy(50, True, False) == 50

    assert subsidy(400, False, True) == 0
```

BranchCoverageTest_PES2UG24CS577_electricity_bill.py

```
import pytest
from electricity_bill import energy_charge, late_fee, subsidy


# Example test cases:
def test_low_usage_domestic():
    assert energy_charge(50, "domestic") == 200


def test_paid_on_time():
    assert late_fee(500, 0) == 0


# --- energy_charge tests ---
def test_energy_charge_unknown_connection():
    # Tests the unknown connection ValueError branch
    with pytest.raises(ValueError, match="unknown connection"):
        energy_charge(50, "industrial")


def test_energy_charge_negative_units():
    # Tests the invalid units ValueError branch
    with pytest.raises(ValueError, match="invalid units"):
        energy_charge(-10, "domestic")


def test_energy_charge_mid_usage():
    # Tests the 101-300 unit slab
    assert energy_charge(200, "domestic") == 850


def test_energy_charge_high_usage_commercial():
    # Tests the >300 unit slab and commercial multiplier
    assert energy_charge(400, "commercial") == 4350


def test_energy_charge_exact_boundaries():
    # Tests exactly 100 and 300 unit boundaries
    assert energy_charge(100, "domestic") == 350
    assert energy_charge(300, "commercial") == 2750
```

```python
# --- late_fee tests ---

def test_late_fee_within_15_days():

    # Tests the 1-15 days branch

    assert late_fee(500, 10) == 50


def test_late_fee_over_15_days_low_bill():

    # Tests >15 days but bill <= 2000

    assert late_fee(1000, 20) == 100


def test_late_fee_over_15_days_high_bill():

    # Tests >15 days and bill > 2000 (triggers additional penalty)

    assert late_fee(2100, 20) == 205


# --- subsidy tests ---

def test_subsidy_senior_low_usage_no_solar():

    # Tests senior discount without solar

    assert subsidy(100, True, False) == 50


def test_subsidy_not_senior_low_usage_solar():

    # Tests solar discount without senior discount

    assert subsidy(100, False, True) == 100


def test_subsidy_senior_high_usage_solar():

    # Tests solar benefit cancellation (units > 300)

    assert subsidy(400, True, True) == 0


def test_subsidy_mid_usage_solar():

    # Tests standard solar benefit (units between 101 and 300)

    assert subsidy(200, False, True) == 100


def test_subsidy_senior_low_usage_solar():

    # Tests combined senior and solar benefits

    assert subsidy(50, True, True) == 150
```